Programming Assignment 3 (PA3)

Magical Demon Hunters Game
User-Defined Types, Encapsulation, Operator Overloading, Interacting Objects
Out: March 11, 2026 -- DUE: April 7, 2026
EC327 Introduction to Software Engineering – Fall 2026

Total: 200 points

- *You may use any development environment you wish, as long as it is ANSI C++ 11 compatible. Please make sure your code compiles and runs properly under the Linux/Unix environment on the PHO 305/307 (or eng-grid) machines before submitting.*
- *PAs may be submitted up to a week late at the cost of a **30% fixed penalty** (e.g., submitting a day late and a week late is equivalent). It is in your best interest to complete as many checkpoints as possible before the deadline. If you have missing implementations in your original submission, you may complete and submit the missing solutions during the following week. Any submissions after the deadline will be subject to the 30% penalty. No credit will be given to solutions submitted after the 1-week late submission period following the deadline.*
- *Follow the assignment submission guidelines in this document or you will lose points.*

Submission Format (Must Read)

- Use the **exact** file names specified in each problem for your solutions.
- Push your changes to the repository generated by GitHub Classroom.
- Complete submissions should have **22 files.**
- Please do **NOT** submit *.exe and *.o or any other files that are not required by the problem.
- **Code must compile in order to be graded. Otherwise, this is an automatic zero.**
- Comment your code (good practice!). We **may** use your comments when grading.

Coding Style (reminder from PA2) As you become an experienced programmer, you will start to realize the importance of good programming style. There are many coding "guidelines" out there and it is important that you begin to adopt your own. Naming conventions will help you recognize variable names, functions, constants, classes etc. Similarly, there are many ways to elegantly format your code so that it is easy to read. All of these issues do not affect compilation and hence are easy to overlook at this stage in your development as a programmer. However, your ability (or inability) to create clean, readable code could be the difference in your career when you leave college.

Good reference: https://google.github.io/styleguide/cppguide.html If you find any other good references, feel free to post the link on Piazza!

Class Specifications

Each class and its members are described below by listing its name, the prototypes for the member functions, and the names and types of the member variables. You **MUST** use the prototypes, types, and names in your program as specified here, in the same upper/lower case. Function argument names are allowed to be different. **Failure to follow these specifications will result in lost points.**

Magical Demon Hunters Game

In this programming assignment, you will be implementing a "simulation" that involves "mage vs demon battles" consisting of objects located in a two-dimensional world that move around and behave in various ways. The user enters commands to tell the objects what to do, and they behave in simulated time. Simulated time advances one "tick" or unit at a time. Time is "frozen" while the user enters commands. When the user commands the program to "go", time will advance one tick of time. When the user commands the program to "run", time will advance several units of time until some significant event happens (to be described later).

How to Play:

You are a mage. You are trying to defeat the demons but you need to have enough mana to do it. You must find demons and earn experience by battling. However, you also need to periodically get magic crystals if you want to make sure you keep your magic powers. You "win" the game by defeating all the demons without losing your magic powers (having 0 mana). You can compare games with other students by seeing who defeats all of the demons the fastest.

In this assignment, you will be implementing these 9 classes:

- `Point2D` – represents a point on a Cartesian coordinate system.
- `Vector2D` – represents a vector in the real plane.
- `GameObject` – base class for all objects in the game.
- `Building` – base class for all building objects in the game.
- `ManaSpire` – The ManaSpire can be used to recover mana but they have a limit on usage. Inherits various member variables from Building.
- `DemonHideout` – location for demons to battle. "Experience" is gained by completing "battles". Inherits various member variables from Building.
- `Mage` – a simulated player object which has three abilities:
  - Move to a specified location

  - Recover mana from a specific ManaSpire
  - Gain experience from a specific DemonHideout

- `View` - Displays game objects.
- `Model` - Holds references to game objects.
- You will also provide a set of separate, related functions combined in one `.cpp` and one `.h` file:
- `GameCommand` - Handles commands from the user input. (2 files)
- You will also be turning in `main.cpp` and a `Makefile`.

Assignment Structure

This assignment is divided into five parts, each with its own learning goals.

Checkpoint 1

In the first checkpoint, you will implement the `Point2D`, `Vector2D`, and `GameObject` classes. You'll get used to writing classes and the associated infrastructure. If you find yourself stuck at all, even for a short time, feel free to post liberally on Piazza.

Checkpoint 2

In this checkpoint, you will be implementing the `Building` class and its associated subclasses (`ManaSpire` and `DemonHideout`). This will start to get you comfortable with the ideas of abstraction and inheritance, as well as start you on the beginning of the design of the actual game. Again, if you find yourself stuck, please post on Piazza or ask during office hours.

Checkpoint 3

In this checkpoint, you will implement the `Mage` class. **Read the instructions carefully**. The `Mage` is functionally the "Player Character" of the game. There are many associated behaviors and interactions, and the majority of the game design will be implemented in this step. Pieces of this may be confusing. If you run into difficulties, please ask for clarification or help on Piazza or office hours.

Checkpoint 4

At this point, it's time to start learning about the MVC (model-view-controller) pattern that is commonly used in software engineering. In particular, you will implement the `Model` class and the `GameCommand` (controller) class. If you want help at this point (or any point), please ask on Piazza or meet with course staff for help (e.g. office hours).

Submission

Now it's time for everything to come together. You will be implementing the `View` piece of the MVC pattern. At this point, you'll have a working game! If your game doesn't work, you guessed it, **ask for help** either on Piazza or in person.